

RMSProp Optimizer

Deep Learning Optimizers — Part 5 | Study Notes

1. What is RMSProp?

Full Form: Root Mean Squared **Propagation**

RMSProp is an adaptive learning rate optimization technique and an improvement over the AdaGrad optimizer. It was introduced to fix AdaGrad's major limitation — the ever-shrinking learning rate problem.

2. Background: The AdaGrad Problem

To understand RMSProp, you first need to understand why AdaGrad fails:

- **Sparse / skewed data issue:** When your dataset has one column of all 1s and another with a wide spread of values (like linear regression with large feature variance), the loss-parameter contour plot becomes very elongated (stretched).
- **Oscillating updates:** During optimization toward the minimum, gradient descent oscillates — moving fast in the bias (b) direction but slow in the weight (w) direction, making convergence slow.
- **Learning rate decay problem:** AdaGrad accumulates ALL past squared gradients from step 1 onward. This sum keeps growing, which makes the effective learning rate shrink to near zero over time. Eventually, updates become so tiny that the algorithm gets stuck — it cannot reach the global minimum.

Root cause: AdaGrad's V_t term considers the entire gradient history from the very first step, causing V_t to grow unboundedly large.

3. AdaGrad Update Rule (for reference)

AdaGrad Parameter Update:

```
theta = theta - (alpha / sqrt(V_t + epsilon)) * gradient
```

```
where V_t = sum of all past squared gradients (from step 1 to t)
```

As training continues, V_t grows very large $\rightarrow$ $\alpha/\sqrt{V_t} \rightarrow \sim 0 \rightarrow$ updates stop $\rightarrow$ algorithm gets stuck.

4. How RMSProp Fixes the Problem

RMSProp makes one small but crucial change: instead of accumulating ALL past squared gradients (as AdaGrad does), it uses an **Exponentially Weighted Moving Average (EWMA)** of past squared gradients.

This means **recent gradients get higher weight** and **older gradients are gradually forgotten**. As a result, V_t never grows too large.

5. The Mathematics of RMSProp

Step 1 — Compute V_t (Exponentially Weighted Average):

$$V_t = \text{beta} * V_{(t-1)} + (1 - \text{beta}) * (\text{gradient}_t)^2$$

Step 2 — Update Parameters:

$$\text{theta}_t = \text{theta}_{(t-1)} - (\text{alpha} / \sqrt{V_t + \text{epsilon}}) * \text{gradient}_t$$

Key Hyperparameters:

Symbol	Name	Typical Value	Description
alpha	Learning rate	0.001	Base step size for updates
beta	Decay rate	0.9	Controls weight given to past gradients
epsilon	Smoothing term	1e-8	Prevents division by zero
V_t	Moving avg.	—	EWMA of squared gradients

6. Why EWMA Prevents V_t From Growing Too Large

Consider steps 1, 2, 3... The contribution of older gradients keeps getting multiplied by beta (e.g. 0.9) each step. So gradient from step 1 eventually has weight 0.9^n — an exponentially small number. As n grows, old gradients contribute almost nothing to V_t .

Step	Contribution of that step's gradient to V_t
Latest (step t)	$(1 - \text{beta}) \times \text{grad}_t^2$ — highest weight
Step $t-1$	$\text{beta} \times (1 - \text{beta}) \times \text{grad}_{(t-1)}^2$
Step $t-2$	$\text{beta}^2 \times (1 - \text{beta}) \times \text{grad}_{(t-2)}^2$
Step $t-k$	$\text{beta}^k \times (1 - \text{beta}) \times \text{grad}_{(t-k)}^2$ — very small

Result: V_t stays bounded → learning rate doesn't shrink to zero → updates keep flowing → algorithm converges!

7. AdaGrad vs RMSProp — Side-by-Side

Aspect	AdaGrad	RMSProp
V_t formula	Sum of ALL past grad^2	EWMA of recent grad^2
V_t growth	Keeps growing → unbounded	Stays bounded
Learning rate	Decays to ~0 over time	Remains meaningful

Convergence	Gets stuck (non-convex)	Reaches global minimum
Best for	Convex problems only	Convex + non-convex (NNs)
Memory of past	Full history (equal weight)	Fades old, keeps recent

8. Advantages of RMSProp

- ✓ Fixes AdaGrad's vanishing learning rate problem via EWMA.
- ✓ Works well on both convex problems (linear regression) and non-convex problems (neural networks).
- ✓ Handles sparse and dense gradients effectively.
- ✓ Was the go-to optimizer for neural networks before Adam was introduced.
- ✓ Still competitive with Adam on many problems — worth trying when Adam doesn't give good results.
- ✓ No known theoretical disadvantages; proven effective in practice.

9. Disadvantages of RMSProp

RMSProp has **no major theoretical disadvantages**. The only practical consideration is:

- The Adam optimizer (which combines RMSProp + Momentum) tends to perform slightly better on many tasks, so RMSProp is used a bit less today — but it remains a very powerful technique.

10. When to Use RMSProp

Scenario	Recommendation
Adam gives poor results	Try RMSProp — often competitive
Training deep neural networks	RMSProp works well (non-convex)
Sparse/skewed input features	RMSProp handles this better than vanilla SGD
Simple convex problems	Both AdaGrad and RMSProp work; RMSProp is safer
State-of-the-art needs	Try Adam first, then RMSProp as strong alternative

11. Quick Recap — Key Takeaways

RMSProp = Root Mean Squared Propagation An adaptive optimizer that improves AdaGrad.

Core idea: Use EWMA of squared gradients instead of the full historical sum.

Effect: V_t stays bounded → effective learning rate never collapses to zero.

Result: Algorithm keeps updating and converges, even on complex non-convex landscapes.

Historical note: Was the standard optimizer for neural networks before Adam arrived.

Today: Still a strong choice; use it when Adam underperforms.
